

Modul Praktikum

Algoritma dan Struktur Data (TPL2106)

Oleh:

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

**Teknologi Rekayasa Perangkat Lunak
Semester Genap
Tahun Akademik 2025/2026**

Modul 5

Tree I :Dasar Tree, CBT, Traversal

1. Deskripsi Singkat Modul

Modul 5 membahas konsep dasar struktur data non-linear yaitu Tree, yang merupakan pengembangan dari struktur data linear yang telah dipelajari sebelumnya. Pada modul ini, mahasiswa akan mempelajari bagaimana data dapat direpresentasikan dalam bentuk hierarki menggunakan node yang saling terhubung.

Materi yang dibahas meliputi konsep dasar Tree, terminologi dalam Tree, Binary Tree, Complete Binary Tree (CBT), serta teknik traversal (penelusuran) Tree seperti preorder, inorder, dan postorder. Selain memahami konsep, mahasiswa juga akan mengimplementasikan struktur Tree menggunakan bahasa pemrograman Python melalui latihan bertahap dan studi kasus.

2. Capaian Pembelajaran

Setelah mengikuti praktikum pada Modul 5 ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar Tree sebagai struktur data non-linear.
2. Mengidentifikasi elemen-elemen dalam Tree (root, parent, child, leaf).
3. Mengimplementasikan Binary Tree menggunakan Python.
4. Memahami dan mengidentifikasi Complete Binary Tree (CBT).
5. Mengimplementasikan metode traversal Tree (preorder, inorder, postorder).
6. Menganalisis perbedaan hasil traversal pada struktur Tree.
7. Menulis kode program Tree secara terstruktur dan terdokumentasi.

3. Prasyarat Pengetahuan Awal

Sebelum mengikuti praktikum pada modul ini, mahasiswa diharapkan telah memahami:

1. Struktur data Linked List
2. Konsep Node dan pointer
3. Stack dan Queue
4. Fungsi dan class dalam Python
5. Perulangan dan percabangan
6. Konsep dasar algoritma

4. Kebutuhan Alat

A. Perangkat Keras

Laptop/PC dengan spesifikasi minimum:

- RAM minimal 4 GB (disarankan 8 GB)
- Ruang kosong minimal 1 GB

Koneksi internet (opsional) untuk instalasi awal dan update extension.

B. Perangkat Lunak

- Python 3.x (disarankan Python 3.10+)
- Visual Studio Code (VS Code)
- Extension VS Code: Python (Microsoft)
- Windows Command Prompt / PowerShell / Git Bash

C. Folder kerja praktikum

5. Instruksi Ketentuan Teknis Praktikum

- a) Mahasiswa wajib menyimpan seluruh file praktikum dalam repository Github masing-masing sesuai dengan ketentuan yang berlaku.
- b) Penamaan file mengikuti instruksi modul
Praktikum9-materi1.py s/d praktikum9-materi6.py
- c) Setiap kode program harus diberi **komentar** untuk menjelaskan tujuan

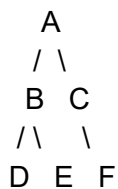
blok program sebagai pembiasaan dokumentasi kode. Serta gunakan komentar pada program terkait untuk menjawab pertanyaan-pertanyaan pada latihan

6. Materi

6.1 Konsep Dasar Tree

Tree adalah struktur data non-linear yang digunakan untuk merepresentasikan hubungan hierarki antar data. Berbeda dengan Linked List yang hanya memiliki satu arah, Tree memiliki cabang (branch). Ilustrasi Tree sebagai berikut :

- A = Root (akar)
- B, C = Child dari A
- D, E = Child dari B
- F = Child dari C



Diawali dengan implementasi node pada tree (ingat materi sebelumnya tentang node):

```

# =====
# Implementasi Node pada Tree
# =====

class Node:
    def __init__(self, data):
        self.data = data      # menyimpan nilai node
        self.left = None     # child kiri
        self.right = None    # child kanan

```

6.2 Terminologi Tree

Perhatikan istilah penting dalam konsep tree berikut ini:

Istilah	Penjelasan
Root	Node paling atas
Parent	Node yang memiliki child
Child	Node turunan
Leaf	Node tanpa child
Subtree	Bagian dari Tree

6.3 Membuat Binary Tree Sederhana

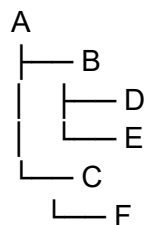
Binary Tree adalah Tree yang setiap node memiliki maksimal 2 child.

```
# Membuat node
root = Node("A")
root.left = Node("B")
root.right = Node("C")

root.left.left = Node("D")
root.left.right = Node("E")

root.right.right = Node("F")
```

Cara membaca strukturnya :



6.4 Konsep Traversal Tree

Traversal adalah proses menelusuri seluruh node dalam Tree. **Jenis Traversal:**

1. **Preorder** → Root → Kiri → Kanan
2. **Inorder** → Kiri → Root → Kanan
3. **Postorder** → Kiri → Kanan → Root

Traversal Preorder

Root → Left → Right

Urutan Preorder:

A → B → D → E → C → F

```
# =====
# Traversal Preorder
# =====
```

```
def preorder(node):
    if node is not None:
        print(node.data)      # 1. Root
        preorder(node.left)  # 2. Kiri
        preorder(node.right) # 3. Kanan
```

Traversal Inorder

Left → Root → Right

Urutan Inorder:

D → B → E → A → C → F

```
# =====
# Traversal Inorder
# =====
```

```
def inorder(node):
    if node is not None:
        inorder(node.left)    # 1. Kiri
        print(node.data)     # 2. Root
        inorder(node.right)  # 3. Kanan
```

Traversal Postorder

Left → Right → Root

Urutan Postorder:

D → E → B → F → C → A

```
# =====
# Traversal Postorder
# =====
```

```
def postorder(node):
    if node is not None:
        postorder(node.left)    # 1. Kiri
        postorder(node.right)  # 2. Kanan
        print(node.data)       # 3. Root
```

6.5 Konsep Complete Binary Tree (CBT)

Complete Binary Tree adalah Binary Tree yang:

- Semua level penuh kecuali level terakhir
- Level terakhir diisi dari kiri ke kanan
- Jadi, jika ada kosong di tengah berarti bukan CBT

```

A
 / \
B   C
 / \ /
D  E F
```

6.6 Perbedaan Linked List dan Tree

Aspek	Linked List	Tree	Keterangan
Struktur	Linear	Non-linear	
Arah	1 arah	Bercabang	Data tidak lagi berurutan tapi bercabang
Node	1 next	left & right	

4. Latihan – Latihan

Bagian ini berisi langkah-langkah praktikum yang disusun secara bertahap untuk membantu mahasiswa memahami konsep tree melalui latihan dan studi kasus.

Latihan 1 . Membuat Node Tree

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
# Membuat node
root = Node("A")
```

```
print(root.data)
```

Checkpoint

- Output menampilkan data A
- left dan right masih bernilai None
- Mahasiswa memahami bahwa node memiliki 2 cabang (left & right)

Latihan 2 . Membuat Binary Tree Sederhana

Struktur Tree yang dibuat :

```
A
 /\
B  C
 /\
D  E
```

Kode Program

```
# =====
# Latihan 2: Membuat Binary Tree Sederhana
# =====

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Membuat root
root = Node("A")

# Membuat child level 1
root.left = Node("B")
root.right = Node("C")

# Membuat child level 2
root.left.left = Node("D")
root.left.right = Node("E")

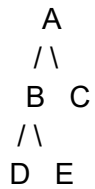
# Menampilkan beberapa hubungan node
print("Root      :", root.data)
print("Child kiri  :", root.left.data)
print("Child kanan :", root.right.data)
print("Child kiri dari B :", root.left.left.data)
print("Child kanan dari B:", root.left.right.data)
```

Checkpoint

- Struktur node berhasil terbentuk
- Mahasiswa memahami hubungan parent dan child

Latihan 3 . Traversal Preorder

Struktur tree yang digunakan



Menelusuri Tree dengan urutan Root → Left → Right

```
# =====
# Latihan 3: Traversal Preorder
# =====

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Fungsi preorder: Root -> Left -> Right
def preorder(node):
    if node is not None:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

# Membuat tree
root = Node("A")
root.left = Node("B")
root.right = Node("C")
root.left.left = Node("D")
root.left.right = Node("E")

# Menjalankan traversal preorder
print("Hasil traversal preorder:")
preorder(root)
```

Output: A B D E C



Latihan 4 . Traversal Inorder

menelusuri Tree dengan urutan Left → Root → Right

```
# =====  
# Latihan 4: Traversal Inorder  
# =====
```

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.left = None  
        self.right = None  
  
# Fungsi inorder: Left -> Root -> Right  
def inorder(node):  
    if node is not None:  
        inorder(node.left)  
        print(node.data, end=" ")  
        inorder(node.right)  
  
# Membuat tree  
root = Node("A")  
root.left = Node("B")  
root.right = Node("C")  
root.left.left = Node("D")  
root.left.right = Node("E")  
  
# Menjalankan traversal inorder  
print("Hasil traversal inorder:")  
inorder(root)
```

Latihan 5 . Melengkapi Fungsi Merge

```
def merge(left, right):  
    result = []  
    i = 0  
    j = 0  
  
    while i < len(left) and j < len(right):  
        if _____:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
  
    result.extend(left[i:])  
    result.extend(right[j:])  
  
    return result
```

Output : D B E A C

Latihan 5 . Traversal Postorder

Menelusuri Tree dengan urutan Left → Right → Root

```
# =====
# Latihan 5: Traversal Postorder
# =====

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Fungsi postorder: Left -> Right -> Root
def postorder(node):
    if node is not None:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")

# Membuat tree
root = Node("A")
root.left = Node("B")
root.right = Node("C")
root.left.left = Node("D")
root.left.right = Node("E")

# Menjalankan traversal postorder
print("Hasil traversal postorder:")
postorder(root)
```

Output : D E B C A

Latihan 6 . Studi Kasus Organisasi dengan StrukturTree

Perhatikan struktur berikut

```
Direktur
├── Manager A
│   ├── Staff 1
│   └── Staff 2
└── Manager B
    └── Staff 3
```

Ketentuan Praktikum

Buatlah semua prgram latihan di atas dan masing-masing latihan disimpan dalam 5 file yang berbeda dengan nama file praktikum9-latihan1.py sampai dengan praktikum9-latihan6.py. Setiap file diberikan header identitas mahasiswa pada bagian



IPB University
— Bogor Indonesia —

Sekolah
Vokasi

atas kode program. Diskusikan kemudian tulis penjelasan dan hasil diskusi pada file program sebagai komentar. Semakin jelas dan lengkap akan semakin bagus. Kumpulkan di akhir praktikum di repo masing-masing pada folder praktikum9.